Defensive Pokémon Team Optimization with Simulated Annealing

Practical Combinatorial Optimization | 11 October 2026
Repository: github.com/phantzzy/Pokemon-Team-Optimziation---PKO

1. Problem formulation

The task is to choose exactly six unique Pokémon from a fixed set of N candidates. Only defensive typing is modeled. Moves, statistics, abilities, items, EVs, IVs, Tera types, usage data, and battle simulation are intentionally excluded.

Let P be the candidate set and T a solution. A valid solution is an unordered six-element subset of P. The search space therefore contains C(N, 6) possible teams. It grows from 924 teams for N=12 to 1,192,052,400 teams for N=100.

For attacking type t and Pokémon p, m(t,p) is the standard defensive effectiveness multiplier. Dual-type effectiveness is the product of the two single-type multipliers.

```
minimize f(T) = sum over p in T and t in Types of m(t,p)
```

Lower cost means better average defensive typing. The human-readable metric is AverageMultiplier = f(T) / 108 because six Pokémon are evaluated against 18 attacking types.

2. Implemented solution

a) Domain

A solution contains six unique candidates. Five fixed, nested instances contain 12, 20, 30, 50, and 100 Pokémon. The local dataset contains 100 species and a complete 18 by 18 type chart derived from PokéAPI. Instance membership, size, and uniqueness are validated.

Instance	N	Possible teams
small_12	12	924
medium_20	20	38,760
medium_30	30	593,775
large_50	50	15,890,700
xlarge_100	100	1,192,052,400

b) Evaluation

Every Pokémon receives a precomputed vector of 18 defensive multipliers. A team evaluation sums six vectors. The calculation is deterministic, independent of team order, and performs no file access inside the optimization loop.

2. Implemented solution - continued

c) Move and neighborhood

A neighbor uses a 1-out / 1-in swap. One current member is selected uniformly, one candidate outside the team is selected uniformly, and the first is replaced by the second. The result always remains a valid six-member team without duplicates.

d) Simulated Annealing

Simulated Annealing was implemented manually without an optimization library. It begins from a seeded random team. For a neighbor, delta is candidate cost minus current cost. Better or equal candidates are always accepted. A worse candidate is accepted probabilistically:

```
P(accept) = exp(-delta / temperature)
```

Temperature is multiplied by alpha after each iteration. The current solution and the best solution seen during the whole run are tracked separately. The run ends at the iteration limit or when temperature falls below the minimum.

Defaults are initial temperature 10.0, alpha 0.995, minimum temperature 0.0001, and at most 20,000 iterations. The temperature condition ends a default run after 2,297 iterations. The same instance, seed, and parameters reproduce the same result.

A separate exact solver enumerates every six-member combination for small instances. It proves the comparison optimum and is not the main optimization method.

3. Testing methodology

The project contains 48 automated unit and integration tests. They cover known type matchups, data validation, deterministic evaluation, neighborhood invariants, SA validity and reproducibility, stopping rules, exact-solver correctness, fixed instances, CLI behavior, gap calculations, and CSV output. All tests passed on Python 3.11.9.

SA was run 30 times per instance using seeds 0 through 29. Every run recorded cost, average multiplier, execution time, seed, and team. Instances through N=30 were also solved exactly. Their gap is:

```
Gap(%) = (SA - OPT) / OPT * 100
```

For N=50 and N=100, exhaustive search was not run. The lowest cost observed across all SA runs is labeled best-known and is not presented as a proven optimum.

Proven exact results

Instance	Teams evaluated	Optimal cost
small_12	924	108.75
medium_20	38,760	104.50
medium_30	593,775	103.50

4. Benchmark results

Instance	Reference	Best	Average	Worst	Avg gap	Avg runtime
small_12	108.75 OPT	108.75	108.75	108.75	0.0000%	9.418 ms
medium_20	104.50 OPT	104.50	104.50	104.50	0.0000%	11.242 ms
medium_30	103.50 OPT	103.50	103.50	103.50	0.0000%	12.652 ms
large_50	101.75 BK	101.75	101.75	101.75	0.0000%	14.534 ms
xlarge_100	93.75 BK	93.75	93.75	93.75	0.0000%	24.019 ms

OPT = proven optimum; BK = best-known observed SA value.

Interpretation

SA reached the proven optimum in all 30 runs for each of the 12-, 20-, and 30-candidate instances. Their best, average, and worst gaps were therefore zero. Every run also returned the same cost for N=50 and N=100. Those values remain best-known because exhaustive search was not performed at those sizes.

Average runtime increased from 9.418 ms for N=12 to 24.019 ms for N=100, remaining practical for interactive use. The consistency is plausible for this deliberately simple additive defensive-cost model, but it does not prove global optimality for the two large instances.

5. Conclusion

The project demonstrates a complete optimization workflow: a precise domain, deterministic objective, validity-preserving neighborhood, custom Simulated Annealing, exact verification on small cases, reproducible experiments, and CSV reporting. The implementation found all known small-instance optima reliably and produced stable best-known results for much larger search spaces.

Data source: PokéAPI local snapshot. Full source code, instances, tests, and benchmark CSV are available in the repository linked above.